

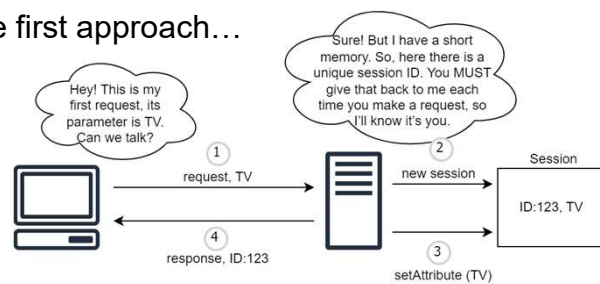
Sessions

- Sessions are higher level mechanisms than cookies
- For each conversation, the **web server** generates a unique ID and a data structure in memory which are associated both with that conversation
- The unique ID is exchanged with the client during its conversation with the web server
- The data structure is used to keep track of the state (i.e., it contains all the information describing the state)

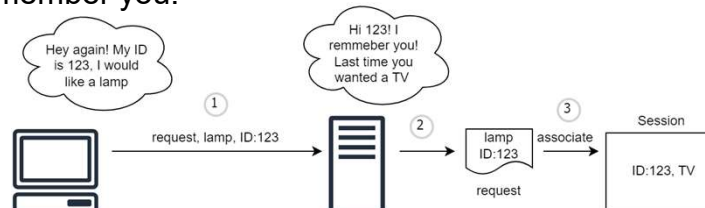
118

Can we talk?

- The first approach...



- I remember you!



119

How does the unique ID is exchanged?

- In all the responses to the client, the web server automatically and transparently adds a cookie with the session ID
- This cookie is called **session cookie** (in Java applications the name of such a cookie is **JSESSIONID**):
 - It has not expiration time
 - It dies at the end of the session
- Through the ID, the web server retrieves the data structure containing the information associated to that session

120

HttpSession

- Sessions are represented by an instance of a class that implements the **HttpSession interface**

HttpSession Interface
(javax.servlet.http.HttpSession)

HttpSession
Object getAttribute(String)
long getCreationTime()
String getId()
long getLastAccessedTime()
int getMaxInactiveInterval()
getServletContext()
void invalidate()
boolean isNew()
void removeAttribute(String)
void setAttribute(String, Object)
void setMaxInactiveInterval(int)
// some other methods

121

Workflow (more or less)

- A Http request arrives at Tomcat
- Tomcat request and response objects (invisible to us)
- Spring starts handling the request
- When a @SessionScope bean is **used** by the web app, Tomcat retrieves or creates the HTTP session:
 - If no session still exists:
 - A HttpSession object is created
 - A JSESSIONID cookie is generated
 - A timeout is configured
 - If the session exists
 - The session is retrieved
- Tomcat returns the HttpSession object to Spring
- Spring creates a @SessionScope bean, binds it to the session, and injects it where needed

122

@SessionScope Beans and HTTP Session

- A bean annotated with @SessionScope is bound to the lifecycle of the HTTP session
- When a method of a Controller performs the first call to a method of a bean annotated as @SessionScope, the bean is created
- The same bean instance is reused throughout the session
- When the session ends (**timeout** or **logout**), the bean is destroyed

Spring does not create or manage the HTTP session itself; it only manages the beans associated with it

123

What is timeout?

- Timeout is the maximum amount of time a HTTP session waits for an event or activity before it is automatically terminated from the server
- The timeout is reset on every user request
- It measures inactivity time, it is not the total session duration!
- Default timeout is 30 minutes (Tomcat decides it)
- Timeout value can be automatically re-configured in Spring Boot in the application.properties file (overriding the default):

```
server.servlet.session.timeout=10m
```

124

At timeout

- Tomcat monitors the activity / inactivity of HTTP sessions
- When a session is inactive for the timeout period, Spring is notified about that, and it will trigger the destruction of the corresponding @Session-scoped bean:
 - If the bean has a @PreDestroy annotated method, such a method is invoked;
 - Reference to the bean is released. Thus, the bean becomes eligible for garbage collection by the JVM

The @PreDestroy annotation marks a method to be executed before the bean is destroyed to release resources

125

What is logout?

- Logout is the process of ending a user's authenticated session, so the user is no longer recognized as logged in
- It is an explicit action by the user (e.g., clicking "Logout")

126

At logout

- Something different is needed to manage logout:
 - Clear the state in your Session-Scoped Bean:
Set the state of the bean to null, this removes the user's information from the session-scoped bean;
 - Invalidate the Entire Session:
Inject the HttpSession object into the Controller managing the logout, this assures to also invalidate the entire HTTP session upon logout. Any other session-related data is also cleared;
 - If the bean has a `@PreDestroy` annotated method, such a method is invoked;
 - Reference to the bean is released. Thus, the bean becomes eligible for garbage collection by the JVM

127

esempio 15

128

Session and cookies: differences

- Sessions are a server-side solution to store information, whereas cookies are a client-side solution that stores information on a local computer
- Sessions can be cookies dependent, whereas cookies are not dependent on sessions
- A session ends when the user closes the browser or logout, whereas cookies expire at a certain prefixed time
- A session can store as much data you want, whereas Cookies have a limited size of 4KB

129



Quiz time!

130

Spot the error!

L'HTTP, di per sé, è un protocollo stateless, ovvero non conserva memoria dello stato nelle sequenze di richieste / risposte scambiate tra client e server. Insieme al campo hidden e alla form, i cookie sono uno dei tre meccanismi che consentono ai client e ai server di collaborare per poter conversare ricordandosi l'uno dell'altro. Un cookie è un piccolo frammento di informazione dotata di un nome, un valore ed alcuni attributi opzionali, quali ad esempio il dominio e la data di scadenza. La dimensione massima di un cookie è 1MB, ogni browser supporta circa 300 cookies in totale. I cookies sono tipicamente impostati dal browser e memorizzati nella memoria del server e permettono di identificare in modo univoco un client. In genere, il server utilizza il contenuto dei cookie per determinare se richieste diverse provengono dallo stesso client e quindi fornire una risposta personalizzata. I cookies sono un meccanismo di tipo server-side. Dal punto di vista tecnico, ci sono due tipi di cookie: i cookie di sessione e i cookie persistenti. Un cookie di sessione è un cookie che contiene l'identificativo della sessione e la cui vita finisce al termine della sessione. I cookie persistenti hanno una vita più lunga: infatti essi sono memorizzati e sopravvivono alle chiusure del browser e ai riavvii del computer. Dal punto di vista legale, la legge italiana distingue i cookies in cookies tecnici e cookies di profilazione. Questi ultimi vengono utilizzati al fine di inviare messaggi pubblicitari in linea con le preferenze manifestate dall'utente nell'ambito della navigazione in rete. Per tale ragione, la normativa prevede che gli utenti siano informati del loro utilizzo e non esprimano il loro consenso.

131

True or False?

- In Spring-based web apps, the Front Controller is typically implemented using a single servlet
- A HTTP request must contains at least a header
- In Spring, a bean is always instantiated manually by the developer
- If two CSS selectors have the same specificity, the cascading algorithm examines their origin
- The @Autowired annotation is used to explicitly create a bean in Spring
- XML uses a set of tags predefined by the W3C

132

Spot the error!

Una caratteristica fondamentale dei CSS è il cascading algorithm, che consente agli stili di sovrascriversi o ereditarsi l'un l'altro. Molti modi di includere e combinare le dichiarazioni del foglio di stile possono infatti causare conflitti e contraddizioni. Un conflitto si verifica quando alla stessa proprietà vengono assegnati valori uguali in più dichiarazioni. In questi casi, esiste un sistema di regole che decide quale delle dichiarazioni di stile in conflitto sarà applicata a un elemento. Un foglio di stile può avere tre origini diverse: user-agent, user e author. Tra queste tre origini, il foglio di stile dello user-agent ha la priorità più bassa. Se viene utilizzato un foglio di stile dell'author, che ha una priorità intermedia, il foglio di stile dello user-agent verrà sovrascritto. La parola chiave !importance modifica l'ordine di priorità, tuttavia il foglio di stile dello user rimane a priorità intermedia. Esiste anche una regola di priorità tra i selettori che viene utilizzata quando sussiste un'equivalenza di origine. Secondo tale regola, per ogni selettore viene calcolato un valore che indica il peso del selettore. Questo valore viene definito specificità. La specificità è espressa come valore numerico e quanto più alto è questo valore, tanto più importante è il selettore, che quindi scavalca qualsiasi selettore concorrente con un valore inferiore. Anche i diversi tipi di selettori hanno specificità diverse. Per esempio, un selettore di classe ha una specificità minore di un selettore di elemento. Se due regole CSS hanno la stessa specificità e sono della stessa origine, si usa un'altra regola chiamata ordine di combinazione.

133